Output:

The transformed dataset will have reduced dimensions while retaining the maximum variance: [[-2.505, -0.144],

[-1.927, -0.265],
[0.046, -0.070],
[2.776, 0.204],
[0.658, 0.034],
[1.558, -0.149],
[0.594, 0.389]]

Insights from PCA

1. **Variance Explained:** The proportion of variance retained by each principal component can be analyzed using the explained_variance_ratio_ attribute in Scikit-Learn:

```
print(pca.explained_variance_ratio_)
```

2. **Dimensionality Reduction:** By focusing on components that capture significant variance, PCA simplifies datasets while retaining essential information.
3. **Applications:** PCA is widely used in image processing, gene expression analysis, and recommendation systems, where high-dimensional data is prevalent.

Advanced Considerations

Hyperparameter Tuning

- **Number of Components:** Choosing the optimal number of components involves balancing variance retention and dimensionality reduction. Plotting a cumulative explained variance graph can help determine the appropriate number:

```
import matplotlib.pyplot as plt
cumulative_variance = pca.explained_variance_ratio_.cumsum() plt.plot(range(1, len(cumulative_variance)+1), cumulative_variance) plt.xlabel('Number of Components') plt.ylabel('Cumulative Explained Variance') plt.title('Explained Variance vs. Number of Components') plt.show()
```

Non-Linear Dimensionality Reduction

For datasets with non-linear structures, techniques like Kernel PCA, t-SNE, or UMAP may be more effective. Kernel PCA extends PCA by